

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1003	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	K Y V Satyanarayana Reddy	Reg. No.:	192472157
Date:	25.08.2026	Duration:	60 minutes

Code of Conduct

I K Y V Satyanarayana reddy certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K Y V Satyanarayana reddy

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.

Problem Statement: The Travel Expense Tracking and Budget Management Application addresses the difficulty of recording, organizing, and monitoring travel expenses when notebooks or spreadsheets are used. Manual calculations can cause errors and make it difficult to know the remaining budget during a trip.

Implemented Solution: The proposed application provides a centralized interface where travelers can create trips, set budgets, record categorized expenses, monitor spending, and view financial summaries. It reduces manual calculations and keeps travel expense information organized.

- Describe the project objectives and key functionalities.

Project Objectives: The main objectives are to simplify travel expense management, improve budget control, reduce calculation errors, and provide accurate financial information to

travelers.

Key Functionalities: The system supports user login, trip creation, budget setting, expense recording and categorization, automatic budget calculation, spending alerts, and expense reports.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.

Repository Structure: The repository is organized into frontend, backend, database, tests, and documentation folders. This separation keeps user-interface code, server-side logic, data management, testing, and project documentation organized.

File Naming: Folders and files use descriptive names such as controllers, routes, models, services, Dockerfile, and README.md. Consistent naming makes files easier to identify and reduces confusion during development.

- Justify how the repository layout supports maintainability and collaboration.

Maintainability: The modular folder structure separates different responsibilities, making it easier to locate, test, modify, and maintain individual components without affecting unrelated modules.

Collaboration: Developers can work on frontend, backend, database, testing, and documentation independently. The organized structure also makes code reviews and Git-based collaboration easier.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.

Readability: The source code should use meaningful variable, function, class, and file names with consistent indentation and formatting so that developers can understand it easily.

Modularity: Separating controllers, routes, models, services, and middleware promotes modularity because each component performs a specific responsibility.

Coding Standards and Documentation: Consistent naming, comments for important logic, reusable functions, and README documentation improve code quality and help new developers understand the project.

- Identify strengths and areas for improvement.

Strengths: The project has a clear modular structure, meaningful feature separation, centralized data management, and support for Git and Docker. These practices improve organization and make future development easier.

Areas for Improvement: The project can be improved by adding more automated tests, stronger input validation, comprehensive API documentation, code review through pull requests, and centralized logging and monitoring.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.

Commit History: The repository should contain small, meaningful commits that represent logical changes such as authentication, trip management, expense tracking, reporting, and Docker configuration.

Branching Strategy: The Main–Development–Feature–Release–Hotfix strategy separates stable code from ongoing development and allows features and urgent fixes to be developed safely.

Repository Maintenance: Regular commits, meaningful branch names, a proper .gitignore file, updated README documentation, and removal of unnecessary files keep the repository clean and maintainable.

- Explain how version control supports collaborative software development.

Collaboration: Git allows developers to work on separate feature branches and merge completed work into the development branch without directly changing stable code.

Version Management: Git records changes through commits and history, allowing developers to identify changes, review previous versions, resolve conflicts, and restore stable versions when necessary.

5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.

Testing Procedure: The application is tested by running the frontend and backend, checking database connectivity, and verifying important functions such as login, trip creation, budget setting, expense recording, calculations, and reports.

Expected Result: Each tested function should produce the expected output without errors, while invalid inputs should be rejected with appropriate messages.

- Present test cases, execution results, and screenshots as evidence.

Test Cases: Test cases should cover valid and invalid login, trip creation, budget entry, expense addition, expense editing, budget calculation, and report generation.

Execution Evidence: The report should include screenshots of the application interface, successful test outputs, Git operations, and Docker execution. Actual screenshots from the

project execution should be inserted as evidence.

6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.

Documentation Review: The README should describe the project, features, repository structure, installation procedure, configuration, dependencies, and usage instructions. Clear documentation helps users and developers understand how to run and maintain the application.

- Suggest improvements to enhance usability and maintainability.

Improvements: The documentation can be improved by adding step-by-step installation instructions, screenshots, API details, troubleshooting guidance, dependency versions, environment-variable explanations, and contribution guidelines.

7. Code Review Findings and Recommendations

- Summarize the overall code review findings.

Overall Findings: The project follows a modular structure suitable for a travel expense management application and uses Git and Docker practices that support organized development. The main areas requiring attention are automated testing, documentation completeness, security validation, and consistent coding practices.

- Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Recommendations: Improve code quality through reusable functions, validation, and consistent standards; strengthen repository management through pull requests and protected branches; expand documentation and automated testing; and introduce future enhancements such as receipt scanning, multi-currency support, bank integration, advanced analytics, CI/CD, and container monitoring.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
	organized.			

Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.
---	---	--	---	---

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25